Konzeptbericht

Status	In Arbeit
Projektname	**Löte**
Projektleiter	Levyn Schneider
Auftraggeber	Beat Walter
Autoren	Levyn Schneider, Tobias Balmer, Roger Widmer, Ivo Graber
Verteiler	Beat Walter, Teammitglieder

Änderungskontrolle, Prüfung, Genehmigung

Version	Datum	Beschreibung, Bemerkung	Name oder Rolle
1.0	04.03	Erste Änderungen	Levyn Schneider, Tobias Balmer, Roger Widmer, Ivo Graber
1.1	11.03	Erste Vervollständigung	Levyn Schneider, Tobias Balmer, Roger Widmer, Ivo Graber
1.2	13.03	Finale Version, kleine Änderungen	Ivo Graber, Tobias Balmer

Definitionen und Abkürzungen

Begriff / Abkürzung	Bedeutung
API	Application Programming Interface (Schnittstelle für Softwarekommunikation)
JWT	JSON Web Token (Token-basiertes Authentifizierungsverfahren)
REST	Representational State Transfer (Architekturstil für Webservices)
CRUD	Create, Read, Update, Delete (Grundlegende Datenbankoperationen)

Referenzen

Referenz	Titel, Quelle
[1] Ticketmaster API	https://developer.ticketmaster.com/products-and-docs/apis/partner/
[2] Ticketmaster Discovery API	https://developer.ticketmaster.com/products-and-docs/apis/international-discovery/v2/

Inhaltsverzeichnis

1 Zusammenfassung ... 4
2 Systemanforderungen ... 5
2.1 Anforderungen an die Funktionalität ... 5
2.2 Anforderungen an die Informationssicherheit und den Datenschutz ... 5
2.2.1 Schützenswerte Daten und deren Behandlung ... 5
2.2.2 Anforderungen ... 6
2.2.3 Betriebliche Vorgaben ... 6
3 Systemarchitektur ... 8
3.1 Systemübersicht ... 8
3.2 Gliederung der Lösung ... 9
3.2.1 Präsentationsschicht ... 10
3.2.2 Applikationsschicht ... 10
3.2.3 Datenzugriffsschicht ... 10
3.2.4 Persistenzschicht ... 10
3.3 Technische Schnittstellen ... 10
3.3.1 Systeminterne Schnittstellen ... 11
3.3.2 Externe Schnittstellen ... 11
3.4 Benutzerschnittstelle ... 12
4 Testkonzept ... 13
5 Tabellenverzeichnis ... 14
6 Abbildungsverzeichnis ... 14

1 Zusammenfassung

Das vorliegende Dokument beschreibt das Konzept der Webapplikation Löte, einer Event-Aggregationsplattform, die im Rahmen des Moduls 306 an der GiBB entwickelt wird. Die Applikation ermöglicht Benutzern, Events verschiedener Ticketplattformen zentral zu durchsuchen, zu filtern und als Favoriten zu speichern. Die Systemarchitektur folgt einem Four-Layer-Modell: Das Frontend wird mit dem Angular Framework umgesetzt, das Backend basiert auf Spring Boot (Java) und stellt REST-Schnittstellen bereit, die Datenpersistenz erfolgt über eine PostgreSQL-Datenbank. Externe Event-Daten werden über APIs wie die Ticketmaster Discovery API und EventFrog eingebunden. Das Product Backlog umfasst zehn User Stories, darunter die Anzeige und Filterung von Events nach Ort, Datum und Kategorie, die Suche nach Eventnamen, die Favoritenverwaltung sowie die direkte Weiterleitung zur Ticketplattform. Im Bereich Informationssicherheit wurden acht Anforderungen definiert, die die drei Grundwerte Vertraulichkeit, Integrität und Verfügbarkeit abdecken, insbesondere den sicheren Umgang mit API-Schlüsseln, Benutzerdaten und Kommunikation über HTTPS.
Das Testkonzept basiert auf den definierten User Stories und sieht für jede Story mindestens einen Testfall vor.

2 Systemanforderungen

2.1 Anforderungen an die Funktionalität

Product Backlog	
Anforderungen / Tätigkeiten	**Aufwands-schätzung (Story Points)**
S: Als Benutzer kann ich auf der Startseite alle verfügbaren Events sehen. A: Die Events werden übersichtlich mit Titel, Datum und Ort angezeigt.	8
S: Als Benutzer kann ich Events nach Ort filtern. A: Ich wähle einen Ort aus und sehe nur Events, die dort stattfinden.	3
S: Als Benutzer kann ich Events nach Datum filtern. A: Ich kann einen Zeitraum wählen und bekomme passende Events angezeigt.	2
S: Als Benutzer kann ich Events nach Kategorie filtern. A: Ich wähle eine Kategorie wie "Konzert" oder "Sport" und sehe nur diese Events.	5
S: Als Benutzer kann ich mehrere Filter kombinieren. A: Die Suchergebnisse zeigen nur Events, die alle gewählten Filter erfüllen.	1
S: Als Benutzer kann ich auf ein Event klicken und Details sehen. A: Eine Detailseite zeigt alle Infos zum Event und einen Link zur Ticketplattform.	3
S: Als Benutzer kann ich Events als Favoriten markieren. A: Markierte Events bleiben gespeichert und sind in einer Favoritenliste abrufbar.	2
S: Als Benutzer kann ich meine Favoritenliste aufrufen. A: Alle gespeicherten Events werden übersichtlich aufgelistet und können entfernt werden.	2
S: Als Benutzer kann ich nach Eventnamen suchen. A: Eine Suchfunktion zeigt passende Events basierend auf meiner Eingabe.	3
S: Als Benutzer kann ich über einen Button direkt zur Ticketseite gelangen. A: Der Button öffnet die Originalplattform in einem neuen Tab zum Ticketkauf.	1

Tabelle 1 Initial Product Backlog

Wir haben ein Akzeptanzkriterium pro Story definiert, also sind sie nicht priorisiert.

2.2 Anforderungen an die Informationssicherheit und den Datenschutz

Löte besitzt eine Benutzerregistrierung also werden zwingend Schützenswerte Daten gespeichert. Also sicher die E-Mail und der Name. Daraus ergeben sich folgende relevanten Punkte:

2.2.1 Schützenswerte Daten und deren Behandlung

Die E-Mails der Benutzer werden in der PostgreSQL-Datenbank gespeichert. Dafür wird die Datenbank durch Authentifizierungsmassnahmen geschützt. Die Passwörter werden mit einem zeitgemässen und sicheren Hashing-Algorithmus encodiert und wären somit auch sicher, wenn sie in die Hände von böswilligen Akteuren gelangen.

Die Favoriten der Benutzer stellen nutzungsbezogene Daten dar, die in der PostgreSQL-Datenbank gespeichert werden. Diese sind zwar nicht direkt personenbezogen, können aber Rückschlüsse auf Interessen einzelner Nutzer zulassen. Die API-Schlüssel für externe Plattformen wie die Ticketmaster Discovery API sind vertraulich zu behandeln und dürfen nicht im Frontend oder im öffentlichen Source Code (GitHub) erscheinen.

2.2.2 Anforderungen

Nr	Anforderung	Kategorie
IS-01	API-Schlüssel und Zugangsdaten für externe Plattformen werden ausschliesslich serverseitig gespeichert (z.B. als Umgebungsvariablen) und niemals im Frontend-Code oder im Git-Repository abgelegt.	Vertraulichkeit
IS-02	Alle externen Links zu Ticketplattformen werden vor der Anzeige validiert, um Phishing oder Weiterleitungen auf schädliche Seiten zu verhindern.	Integrität
IS-03	Die Kommunikation zwischen Frontend und Backend erfolgt ausschliesslich über HTTPS.	Vertraulichkeit
IS-04	Benutzereingaben (Suchbegriffe, Filterparameter) werden sowohl im Frontend als auch im Backend validiert und sanitisiert, um Injection-Angriffe (SQL-Injection, XSS) zu verhindern.	Integrität
IS-05	Bei Ausfall einer externen API werden keine sensiblen Fehlermeldungen (Stack Traces, interne Pfade) an den Benutzer weitergegeben. Es wird eine benutzerfreundliche Fehlermeldung angezeigt.	Vertraulichkeit
IS-06	Die Datenbank wird regelmässig gesichert (Backup), um Datenverlust zu vermeiden.	Verfügbarkeit
IS-07	Eine `.gitignore`-Datei stellt sicher, dass Konfigurationsdateien mit Zugangsdaten nicht in das GitHub-Repository gelangen.	Verfügbarkeit
IS-08	Die REST-Schnittstellen des Backends werden durch geeignete CORS-Konfiguration abgesichert, sodass nur das eigene Frontend Anfragen stellen kann.	Integrität

Tabelle 2 Anforderungen

2.2.3 Betriebliche Vorgaben

Da das Projekt im Rahmen des Moduls 306 an der GiBB durchgeführt wird, gelten die schulischen Vorgaben zur Nutzung von Infrastruktur (Teams-Kanal, GitHub). Der ISDS-Verantwortliche Roger Widmer stellt sicher, dass diese Anforderungen während der Umsetzung eingehalten werden.

Die Anforderungen orientieren sich an den drei Grundwerten der Informationssicherheit (Vertraulichkeit, Integrität, Verfügbarkeit) und sind bewusst auf den Projektumfang zugeschnitten – also realistisch für ein Schulprojekt ohne Login-System, aber trotzdem professionell formuliert. Soll ich etwas ergänzen oder anpassen?

2.3 Fachliche Entitätstypen

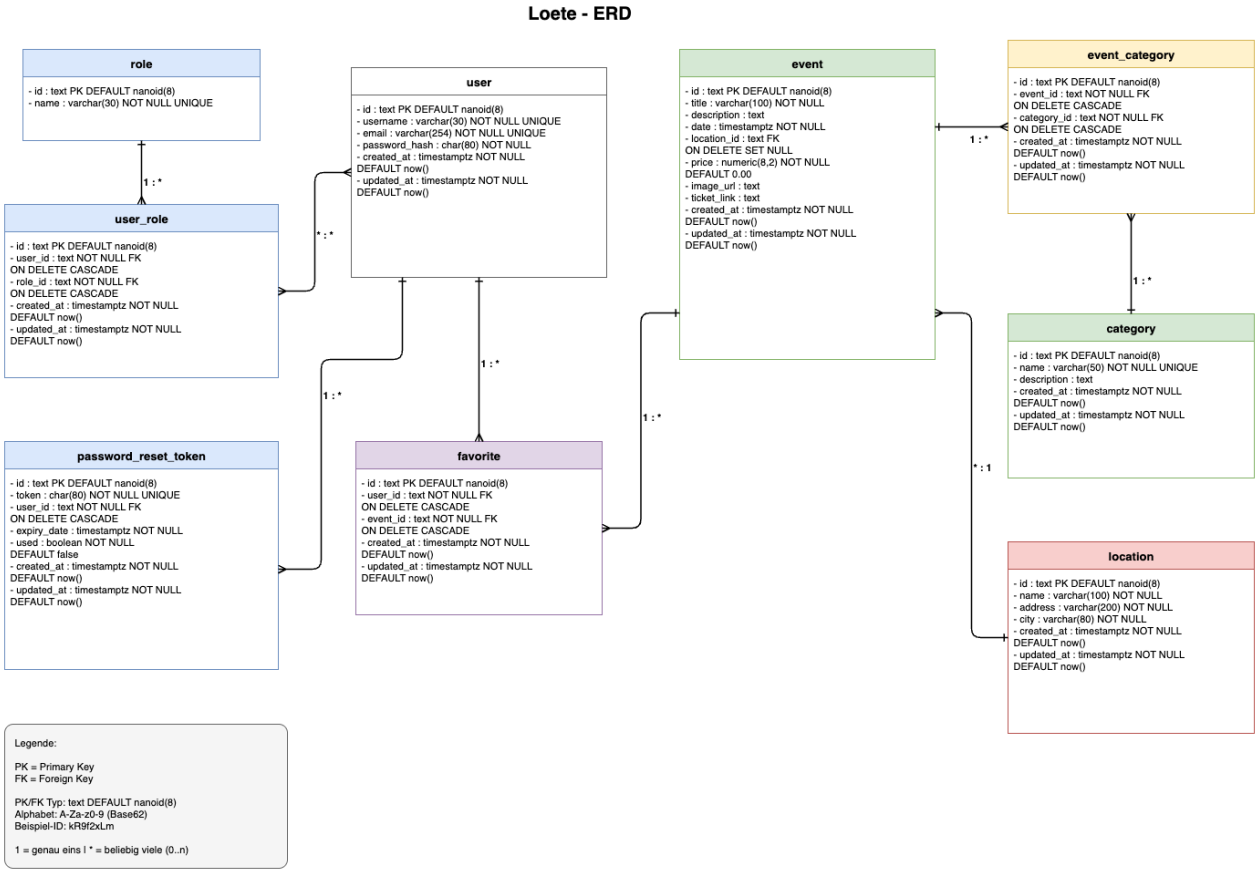

Abbildung 1 Fachliche Entitätstypen

3 Systemarchitektur

Dieses Kapitel beschreibt die Systemarchitektur des Systems Löten.

3.1 Systemübersicht

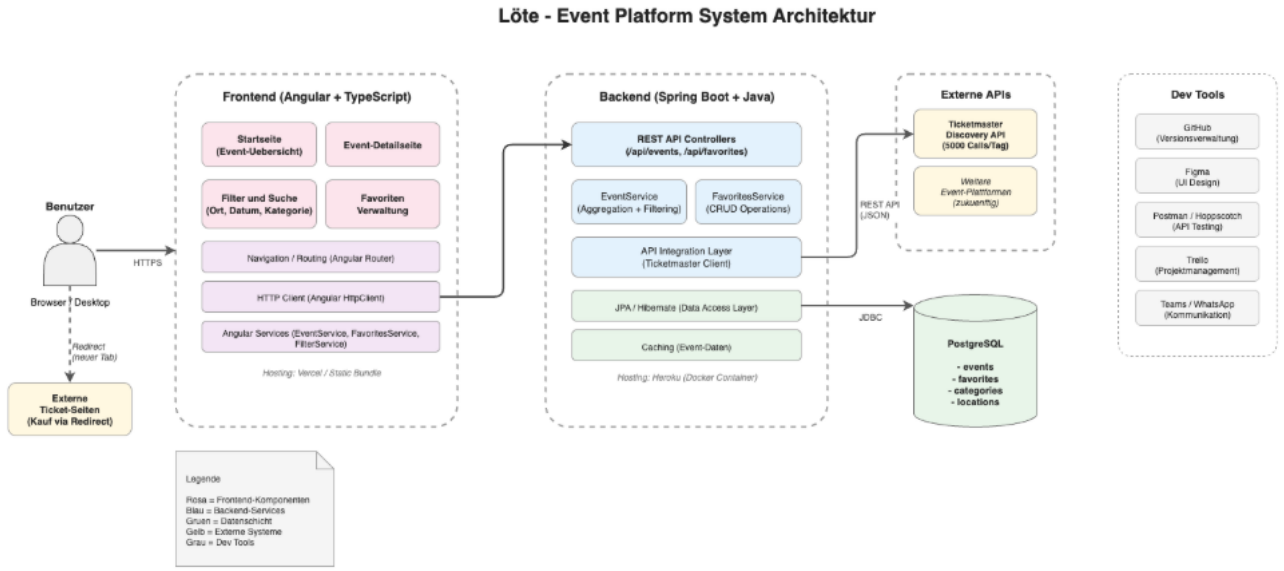

Abbildung 2 Systemarchitektur

Die Lösung besteht aus einem Frontend, einem Backend, einer Datenbank sowie der Anbindung externer Schnittstellen.

Das Frontend wird mit dem Angular Framework auf TypeScript als Single Page Application umgesetzt. Es stellt die Benutzeroberfläche bereit und läuft im Browser auf allen Geräten. Die Anwendung umfasst eine Startseite mit Event Übersicht, eine Event Detailseite, eine Filter- und Suchfunktion nach Ort, Datum und Kategorie sowie eine Favoriten Verwaltung. Die Navigation erfolgt über den Angular Router. Über den integrierten HTTP-Client kommuniziert das Frontend ausschliesslich mit dem Backend.

Das Backend basiert auf dem Spring Boot Framework auf Java und stellt REST-Schnittstellen im JSON-Format bereit. Es enthält die gesamte Geschäftslogik der Anwendung. Über Controller werden Endpunkte wie /events und /favorites bereitgestellt. In der Service Schicht werden Event Daten aggregiert, gefiltert und Favoriten verwaltet. Eine eigene Integrationsschicht bindet externe Event APIs wie die Ticketmaster Discovery API an.

Die Entwicklung wird durch Tools wie GitHub für Versionsverwaltung, Figma für das UI-Design sowie Postman oder Hoppscotch für API-Tests unterstützt.

3.2 Gliederung der Lösung

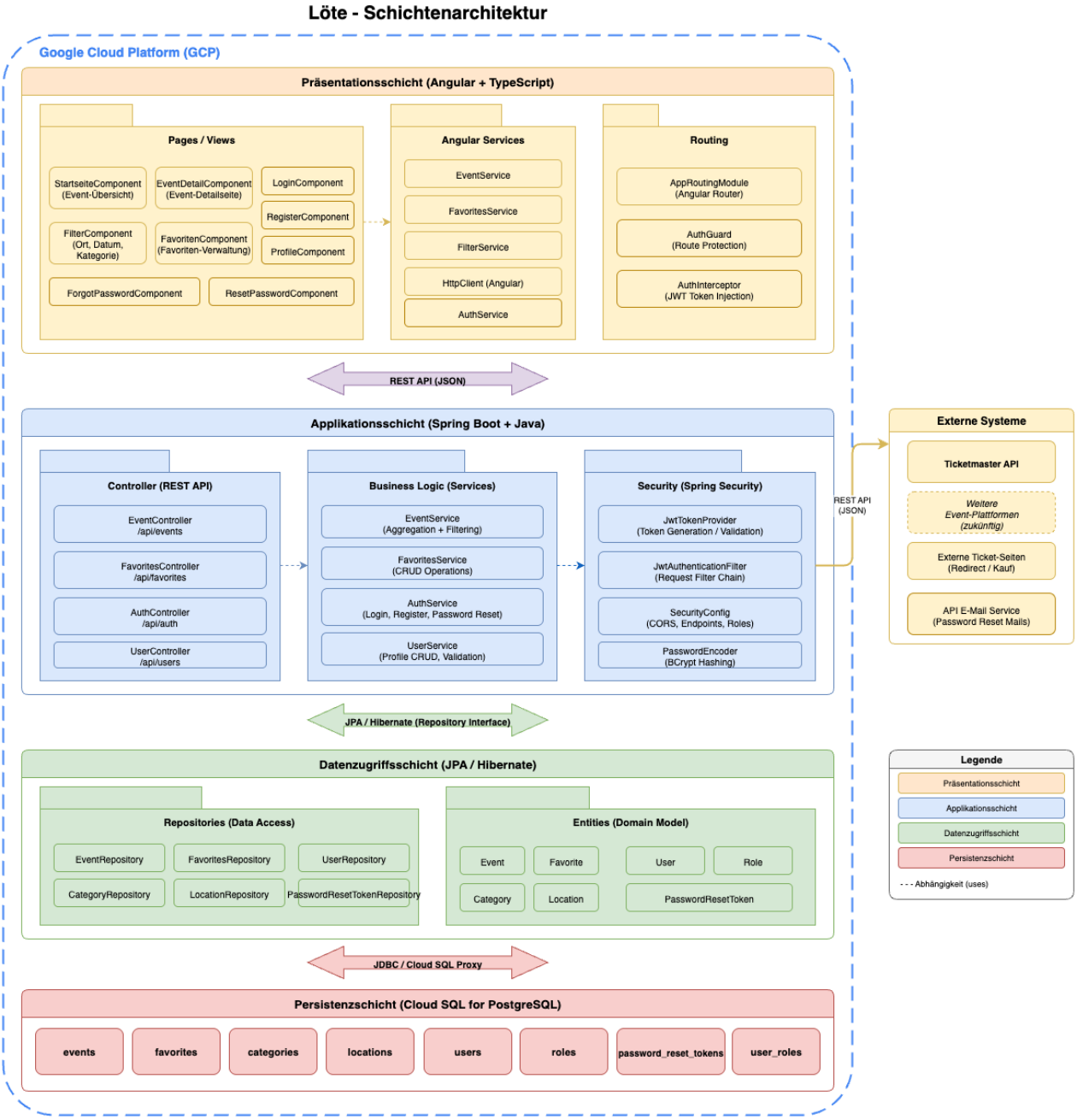

Abbildung 3 Schichtenmodell

Die Applikation ist nach einem Vier-Schichten-Modell aufgebaut und wird auf der Google Cloud Plattform (GCP) betrieben. Sie besteht aus einer Präsentationsschicht, einer Applikationsschicht, einer Datenzugriffsschicht und einer Persistenz Schicht.

3.2.1 Präsentationsschicht

Die Präsentationsschicht gliedert sich in drei Bereiche:

Pages / Views:
- StartseiteComponent – Übersicht aller Events (Feed)
- EventDetailComponent – Detailansicht mit erweiterten Informationen zu einem einzelnen Event
- FilterComponent – Filterung nach Ort, Datum und Kategorie
- FavoritenComponent – Verwaltung der vom Benutzer favorisierten Events
- LoginComponent / RegisterComponent – Benutzeranmeldung und -registrierung
- ProfileComponent – Benutzerprofilverwaltung
- ForgotPasswordComponent / ResetPasswordComponent – Passwort-Zurücksetzung

Angular Services:
EventService, FavoritesService, FilterService, AuthService sowie der HttpClient für die Kommunikation mit dem Backend.

Routing & Sicherheit:
Das AppRoutingModule steuert die Navigation. Ein AuthGuard schützt bestimmte Routen vor unauthentifiziertem Zugriff, und ein AuthInterceptor fügt bei jeder HTTP-Anfrage automatisch das JWT-Token hinzu.

3.2.2 Applikationsschicht

Die Applikationsschicht wird mit Spring Boot (Java) realisiert und gliedert sich in drei Bereiche:

Controller (REST API):
- EventController (/events) – Bereitstellung und Filterung von Events
- FavoritesController (/favorites) – CRUD-Operationen für Favoriten
- AuthController (/auth) – Login, Registrierung und Passwort-Reset
- UserController (/users) – Profilverwaltung

Business Logic (Services):
- EventService – Aggregation von Events aus externen APIs und Filterung
- FavoritesService – CRUD-Operationen für die Favoritenverwaltung
- AuthService – Login, Registrierung und Passwort-Reset-Logik
- UserService – Profilverwaltung und Validierung

Security (Spring Security):
Die Authentifizierung erfolgt über JWT. Der JwtTokenProvider generiert und validiert Tokens, der JwtAuthenticationFilter prüft eingehende Requests. Die SecurityConfig definiert CORS-Einstellungen, geschützte Endpunkte und Rollenberechtigungen. Passwörter werden mit BCrypt gehasht.

3.2.3 Datenzugriffsschicht
Die Datenzugriffsschicht wird mit JPA / Hibernate realisiert und bildet die Brücke zwischen der Applikationslogik und der Datenbank.

Repositories (Data Access): EventRepository, FavoritesRepository, UserRepository, CategoryRepository, LocationRepository, PasswordResetTokenRepository

Entities (Domain Model): Event, Favorite, User, Role, Category, Location, PasswordResetToken

3.2.4 Persistenzschicht
Die Persistenzschicht wird durch eine Cloud SQL for PostgreSQL-Datenbank auf GCP realisiert. Die Verbindung erfolgt über JDBC / Cloud SQL Proxy. Folgende Tabellen existieren: events, favorites, categories, locations, users, roles, password_reset_tokens, user_roles.

3.3 Technische Schnittstellen

3.3.1 Systeminterne Schnittstellen

Frontend ↔ Backend (REST API / JSON):
Die Kommunikation erfolgt über eine REST-Schnittstelle im JSON-Format. Alle Anfragen authentifizierter Benutzer enthalten ein JWT-Token im Authorization-Header. Relevante HTTP-Statuscodes sind u.a. 200 (Erfolg), 401 (nicht authentifiziert), 403 (nicht autorisiert) und 404 (Ressource nicht gefunden).

Folgende Endpunkte stehen zur Verfügung:

- POST /auth/register – Benutzerregistrierung
- POST /auth/login – Benutzeranmeldung
- POST /auth/forgot-password – Passwort-Reset anfordern
- POST /auth/reset-password – Passwort zurücksetzen
- GET /events – Alle Events abrufen (mit Filterparametern)
- GET /events/{id} – Detailansicht eines Events
- GET /favorites – Favoriten des Benutzers abrufen
- POST /favorites – Favorit hinzufügen
- DELETE /favorites/{id} – Favorit entfernen
- GET /users/profile – Benutzerprofil abrufen
- PUT /users/profile – Benutzerprofil aktualisieren

Backend ↔ Datenbank (JPA / Hibernate):
Die Kommunikation mit der PostgreSQL-Datenbank erfolgt über JPA/Hibernate Repository-Interfaces und JDBC via Cloud SQL Proxy.

3.3.2 Externe Schnittstellen

Ticketmaster Discovery API:
Die primäre Quelle für Event-Daten. Die Kommunikation erfolgt über REST API im JSON-Format. Die API-Dokumentation ist unter der offiziellen Ticketmaster Developer-Seite verfügbar.

Weitere Event-Plattformen (zukünftig):
Die Architektur ist so konzipiert, dass weitere Plattformen angebunden werden können.

Externe Ticket-Seiten:
Für den Ticketkauf werden Benutzer per Redirect auf die jeweilige externe Verkaufsplattform weitergeleitet.

API E-Mail Service:
Für den Versand von Passwort-Reset-Mails wird ein externer E-Mail-Dienst angebunden.

3.4 Benutzerschnittstelle

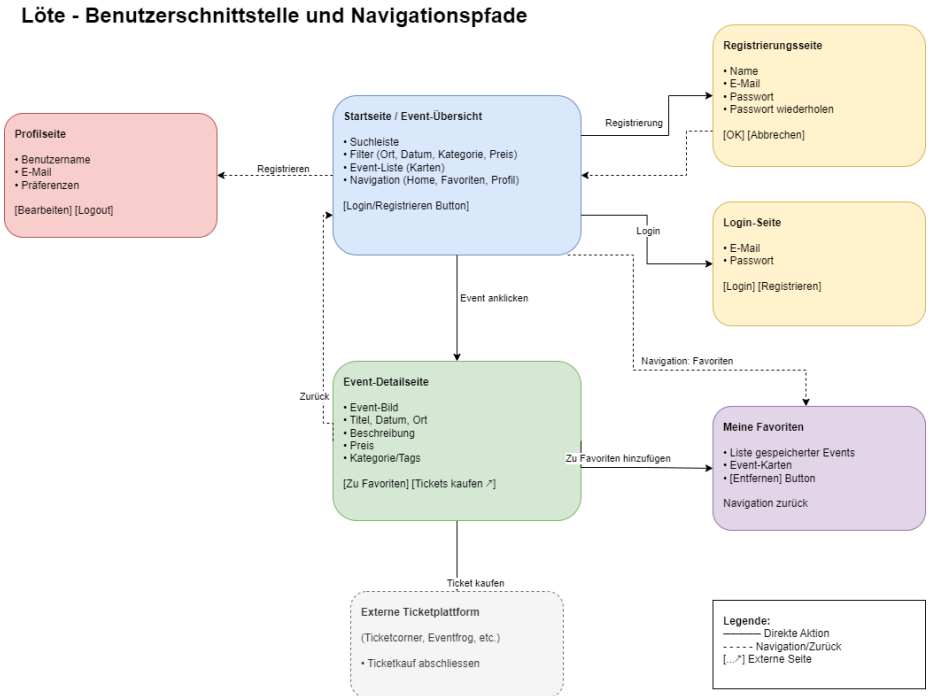

Abbildung 4 Benutzerschnittstellen

Die Benutzerschnittstelle von Löte folgt einem klassischen Navigationskonzept mit der zentralen Startseite als Haupteinstiegspunkt, von der aus Benutzer Events durchsuchen, filtern und direkt zu Detailseiten navigieren können. Nach einer optionalen Registrierung oder Login stehen zusätzliche Funktionen wie das Speichern von Favoriten und die Profilverwaltung zur Verfügung, während der eigentliche Ticketkauf über einen Link zur jeweiligen externen Ticketplattform erfolgt.

4 Testkonzept

Wir testen aufgrund der User Stories. Für jede User Story soll es mindestens einen Test geben. Die Tests sind nach dem jeweiligen Akzeptanzkriterium geschrieben.

Abgedeckte User Stories	Beschreibung
Neues Benutzerkonto erstellen	Ein Testbenutzer, welcher noch kein Konto besitzt, registriert sich beim System und kann sich anschliessend anmelden.
Events auf der Startseite anzeigen	Als Benutzer kann ich auf der Startseite alle verfügbaren Events sehen. Die Events werden übersichtlich mit Titel, Datum und Ort angezeigt.
Events nach Ort filtern	Als Benutzer kann ich Events nach Ort filtern. Ich wähle einen Ort aus und sehe nur Events, die dort stattfinden.
Events nach Datum filtern	Als Benutzer kann ich Events nach Datum filtern. Ich kann einen Zeitraum wählen und bekomme passende Events angezeigt.
Events nach Kategorie filtern	Als Benutzer kann ich Events nach Kategorie filtern. Ich wähle eine Kategorie wie „Konzert“ oder „Sport“ und sehe nur diese Events.
Mehrere Filter kombinieren	Als Benutzer kann ich mehrere Filter gleichzeitig verwenden. Die Suchergebnisse zeigen nur Events, die alle gewählten Filter erfüllen.
Eventdetails anzeigen	Als Benutzer kann ich auf ein Event klicken und Details sehen. Eine Detailseite zeigt alle Informationen zum Event sowie einen Link zur Ticketplattform.
Events als Favoriten markieren	Als Benutzer kann ich Events als Favoriten markieren. Markierte Events bleiben gespeichert und können später wieder aufgerufen werden.
Favoritenliste anzeigen	Als Benutzer kann ich meine Favoritenliste aufrufen. Alle gespeicherten Events werden übersichtlich angezeigt und können auch wieder entfernt werden.
Nach Eventnamen suchen	Als Benutzer kann ich nach Eventnamen suchen. Eine Suchfunktion zeigt passende Events basierend auf meiner Eingabe an.
Zur Ticketseite wechseln	Als Benutzer kann ich über einen Button direkt zur Ticketseite gelangen. Der Button öffnet die Originalplattform in einem neuen Tab zum Ticketkauf.

Tabelle 3 Testkonzept

5 Tabellenverzeichnis

Tabelle 1 Initial Product Backlog 5
Tabelle 2 Anforderungen 6
Tabelle 3 Testkonzept 13

6 Abbildungsverzeichnis

Abbildung 1 Fachliche Entitätstypen 7
Abbildung 2 Systemarchitektur 8
Abbildung 3 Schichtenmodell 9
Abbildung 4 Benutzerschnittstellen 12